

Medical AI Prediction System (MAPS)

A comprehensive, role-based medical platform integrating ML.NET disease prediction, ONNX-powered image classification, NLP-driven clinical note processing, personalized AI chatbot with multi-modal inputs, and Docker containerization within a unified healthcare ecosystem governed by hierarchical access control.

SUBMITTED BY

Muhammad Ameer Hamza · Muhammad Jazib Asad Kayani · Muhammad Abdullah Khan

SUBMITTED TO

Sir Uzair Janjua · Advanced Programming

CLASS

BSCS-6-B · KICSIT Computer Science Department

DATE

May 14, 2026

AI CHATBOT

DOCKER

MULTI-MODAL

ALL OPEN SOURCE

Table of Contents

1. Introduction

1.1 Project Objectives

2. Role-Based Hierarchy System

2.1 Role Feature Comparison

2.2 Chat Permission Matrix

3. System Architecture

3.1 Architecture Diagram

3.2 Architecture Layers

3.3 Technology Stack

4. Module A: Admin Panel

5. Module B: Doctor Module (Enhanced)

5.1 AI-Powered Clinical Workflow

5.2 Doctor Dashboard

5.3 AI Disease Prediction Engine

5.4 AI-Assisted Differential Diagnosis

5.5 Patient Risk Assessment and Proactive Alerts

5.6 Clinical Notes with AI Summarization

5.7 Medical Image Analysis

5.8 Smart Appointment Priority Engine

5.9 Patient Management

5.10 Prescription Management

5.11 Doctor-Patient Chat System

5.12 Medical Reports Generation

5.13 AI Personalized Chatbot for Doctors

5.14 Voice-to-Text Clinical Dictation

5.15 Drug Interaction AI Checker

5.16 Medical Literature Search (RAG)

6. Module C: Patient Module

7. AI/ML Pipeline

8. AI Chatbot Architecture

9. Database Design

10. Docker Containerization

- [11. Security Architecture](#)
- [12. API Design](#)
- [13. Scope and Expected Outcomes](#)
- [14. Conclusion](#)

1. Introduction

The Medical AI Prediction System (MAPS) is a comprehensive, role-based medical platform designed to integrate artificial intelligence into healthcare delivery while maintaining strict professional governance and clinical oversight. This enhanced proposal presents the design, architecture, and implementation plan for MAPS, a system that combines ML.NET-based disease prediction, ONNX-powered medical image classification, NLP-driven clinical note processing, real-time SignalR communication, and a personalized AI chatbot for doctors with multi-modal input support within a unified platform governed by a hierarchical access control model.

MAPS provides two complementary frontends: a WinForms desktop application for clinic-based staff and an ASP.NET Core MVC web application for remote access, both powered by a single shared ASP.NET Core Web API backend. The entire system is containerized using Docker with Docker Compose orchestration, ensuring consistent deployment across development, testing, and production environments. Every component of the technology stack is built on free, open-source tools, eliminating licensing costs and vendor lock-in while maintaining enterprise-grade capabilities through PostgreSQL, Redis, MinIO, Ollama, and Grafana Loki.

MAPS addresses a critical gap in healthcare technology: the absence of unified platforms that combine AI-driven diagnostics with role-appropriate access controls. Existing solutions typically offer either AI prediction capabilities or hospital management features, but rarely both within a single integrated ecosystem. Our platform bridges this divide by providing a layered architecture where Administrators hold the highest authority for system governance, Doctors serve as the clinical backbone with advanced AI-assisted tools including a personalized AI chatbot, and Patients interact through a controlled, secure portal. Critically, the Patient Module does not include direct AI chatbot access; all AI-generated insights are routed through assigned doctors to maintain clinical accountability and prevent misinterpretation of diagnostic results.

1.1 Project Objectives

The primary objectives of the MAPS project are to develop a functional medical AI platform that demonstrates the integration of multiple AI paradigms within a clinically responsible framework. The following table outlines the core project objectives and their associated success criteria.

Objective	Description	Success Criteria
Multi-Disease Prediction	Predict Diabetes, Heart Disease, Pneumonia, Brain Tumour, and Skin Cancer through structured, image, and text input modalities	Prediction accuracy exceeding 85% on validation datasets with confidence scores displayed
AI Chatbot for Doctors	Personalized AI assistant supporting text, image, voice, and document inputs with RAG-enhanced clinical context	Doctors can query the chatbot in 4 input modalities and receive context-aware clinical suggestions with disclaimers
Differential Diagnosis	Generate ranked lists of possible diagnoses with reasoning chains and confirmatory test recommendations	Top-3 differential matches clinical consensus in at least 80% of test cases
Risk Stratification	Continuous patient risk scoring with four-tier urgency classification and proactive alerting	High-risk patients identified with 90%+ recall; alerts delivered within 60 seconds
Smart Scheduling	Prioritize appointments based on AI risk scores for optimal resource allocation	High-risk patients receive same-day appointments
Clinical NLP	Automatic summarization and entity extraction from free-text clinical notes	Entity extraction F1 score above 0.80 on clinical documentation
Docker Deployment	Fully containerized deployment with Docker Compose for one-command infrastructure setup	Complete system deployable via 'docker compose up' with all services healthy within 5 minutes
Real-Time Communication	SignalR-powered chat between doctors and patients with file sharing and read receipts	Message delivery latency under 500ms with persistent history

Table 1: MAPS Project Objectives and Success Criteria

2. Role-Based Hierarchy System

The MAPS platform implements a strict three-tier role hierarchy that governs every aspect of system access, data visibility, and feature availability. This hierarchical model is a foundational architectural principle ensuring clinical accountability, data security, and operational integrity. Administrators possess unrestricted system oversight and control, Doctors serve as clinical decision-makers with advanced AI tools including the personalized AI chatbot, and Patients interact through a controlled portal providing health information without direct AI diagnostic access.

The decision to restrict the AI chatbot to the Doctor role only is clinically motivated. Allowing patients to interact directly with AI diagnostic tools without professional mediation introduces medical liability risks, potential for misinterpretation of results, and the danger of self-diagnosis leading to harmful health decisions. By routing all AI-generated insights through assigned doctors, MAPS ensures that every prediction, risk assessment, and diagnostic suggestion is contextualized and validated by a qualified medical professional before reaching the patient. The AI chatbot serves as a clinical decision support tool, augmenting the doctor's expertise rather than replacing it.

2.1 Role Feature Comparison

Feature / Capability	Admin (Tier 1)	Doctor (Tier 2)	Patient (Tier 3)
User Management (CRUD)	Full Access	View Own Profile	View Own Profile
Doctor-Patient Assignment	Full Control	View Assignments	View Assignment
System Analytics Dashboard	Full Access	Department Stats	No Access
AI Model Administration	Full Control	Use Models	No Access
AI Disease Prediction	View All Predictions	Create + View Own	View Doctor-Shared Only
Differential Diagnosis	View All	Create + View Own	View Doctor-Shared Only
Risk Assessment	View All Scores	View Assigned Patients	View Own Score
Clinical Notes	View All	Create + View Assigned	No Access
Medical Image Analysis	View All	Upload + Analyze	No Access
AI Chatbot (Doctor Only)	No Access	Full Access (4 Modalities)	No Access
Smart Appointment Scheduling	Override Priority	View Priority Queue	Book + Reschedule
Doctor-Patient Chat	Chat with Anyone	Chat with Assigned	Chat with Assigned
Prescription Management	View All	Create and Manage	View Own
Feedback System	View All Feedback	View Own Feedback	Submit Feedback
Health Records	View All Records	View Assigned Records	View Own Records
Docker Deployment Config	Full Access	No Access	No Access

Table 2: Role-Based Feature Comparison Matrix (AI Chatbot highlighted)

2.2 Chat Permission Matrix

Initiator	Can Chat With Admin	Can Chat With Doctor	Can Chat With Patient
Admin	Yes (All Admins)	Yes (Any Doctor)	Yes (Any Patient)
Doctor	Yes (Any Admin)	Yes (Assigned Colleagues)	Yes (Assigned Patients Only)

Initiator	Can Chat With Admin	Can Chat With Doctor	Can Chat With Patient
Patient	Yes (Support Channel)	Yes (Assigned Doctor Only)	No

Table 3: Chat Permission Matrix by Role

3. System Architecture

The MAPS platform follows a layered architecture pattern that separates concerns across presentation, business logic, AI/ML processing, chatbot orchestration, and data persistence layers. This architecture ensures modularity, scalability, and maintainability while supporting the complex interactions between AI models, the multi-modal AI chatbot, real-time communication, and role-based access control. The entire system is containerized using Docker with Docker Compose orchestration, providing isolated network tiers for frontend, backend, AI, and monitoring services.

3.1 Architecture Diagram

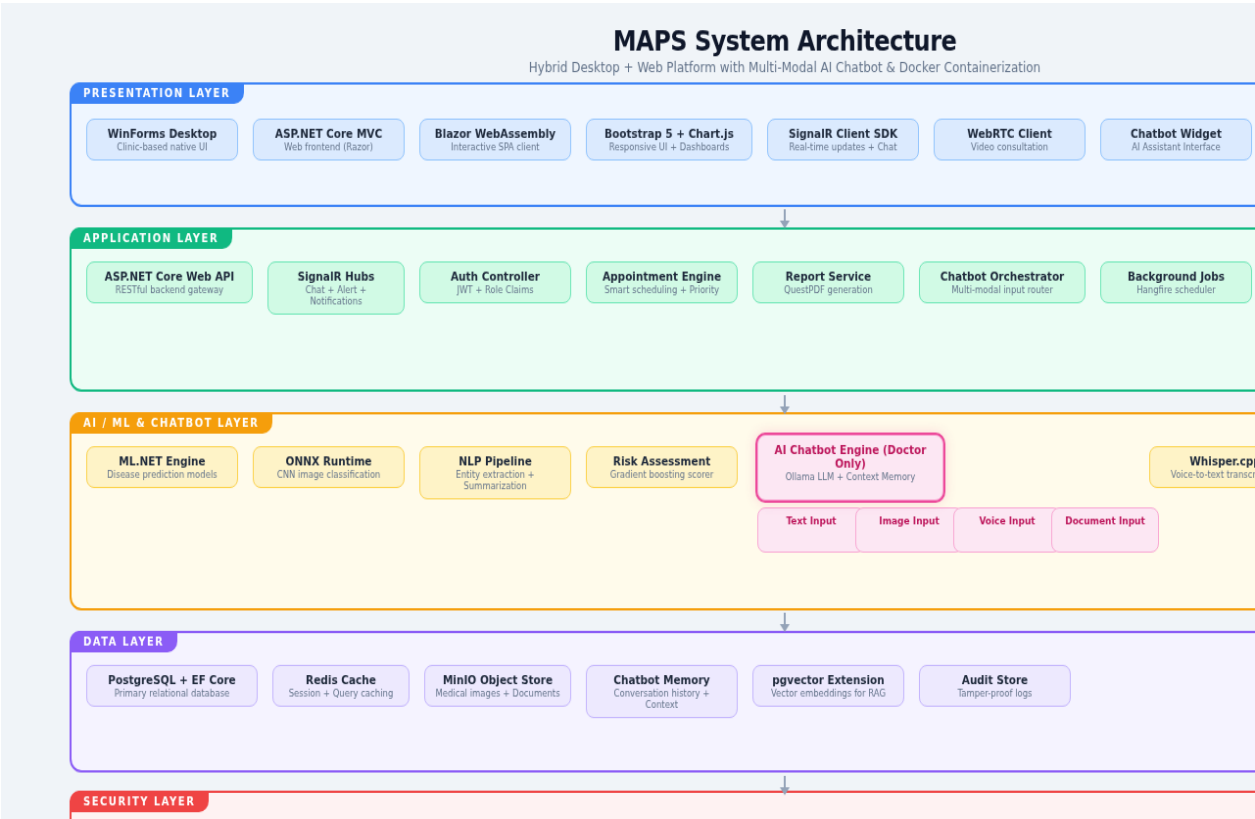


Figure 1: MAPS System Architecture — Six-Layer Design with Docker Containerization and AI Chatbot Layer

3.2 Architecture Layers

Layer	Components	Responsibility
Presentation Layer	ASP.NET Core MVC / Razor Pages, WinForms Desktop, Blazor WebAssembly, Bootstrap 5, Chart.js, SignalR Client SDK, Chatbot Widget	Server-side rendered UI with responsive design for web; native Windows desktop for clinic staff; interactive SPA client; AI chatbot interface for doctors
Application Layer	ASP.NET Core Web API, SignalR Hubs, Controllers, Services, Chatbot Orchestrator, Hangfire Jobs	RESTful API endpoints, business logic orchestration, real-time WebSocket communication, multi-modal chatbot routing, background job scheduling

Layer	Components	Responsibility
AI / ML & Chatbot Layer	ML.NET, ONNX Runtime, Ollama LLaMA 3, Whisper.cpp, NLP Custom Pipeline, pgvector RAG	Disease prediction, image classification, multi-modal chatbot with text/image/voice/document inputs, clinical note summarization, voice transcription, medical literature search
Data Layer	PostgreSQL + EF Core + pgvector, Redis Cache, MinIO Object Store, Chatbot Memory	Relational data persistence with ORM, vector similarity search for RAG, session management, medical image/document storage, conversation history
Security Layer	ASP.NET Core Identity + JWT, TLS 1.3, AES-256, Serilog + Grafana Loki	Secure authentication with role-based claims, end-to-end encryption, data at rest encryption, structured logging with alerting
Docker Layer	Docker Compose, Nginx, Multiple containers, Persistent volumes, Health checks	Container orchestration, SSL termination, network isolation, data persistence, service monitoring

Table 4: MAPS System Architecture — Layered Overview

3.3 Technology Stack

The following table presents the complete technology stack for the MAPS platform. Every technology listed is free and open-source, ensuring zero licensing costs and full community support. Each technology is organized by functional layer with its specific purpose and the rationale for its selection, ensuring transparency in architectural decisions.

Technology	Purpose	Rationale
C# 12 / .NET 8	Primary language and runtime for all backend and frontend components	Strongly typed, modern OOP with full ecosystem support; native integration with ML.NET and ONNX Runtime; completely free and open-source
ASP.NET Core Web API	Shared backend RESTful API serving both desktop and web clients	Industry-standard REST API framework for .NET; unified backend ensures consistent behaviour and data across all clients; free and cross-platform
ASP.NET Core MVC / Razor Pages	Web frontend with server-side rendered views	Razor views in the same C# ecosystem; runs in browser for remote access without desktop installation; free and open-source
WinForms (.NET 8)	Native Windows desktop application for clinic-based staff	Lightweight native desktop UI framework included in .NET SDK at no cost; familiar WinForms designer for rapid clinic UI development
Blazor WebAssembly	Interactive SPA client for real-time features	Runs C# in browser via WebAssembly; shares code with backend; free and open-source; no JavaScript framework needed
Bootstrap 5 + Chart.js	Responsive UI framework and interactive data visualization dashboards	Most popular free CSS framework; Chart.js is MIT-licensed and supports bar, line, pie, heatmap visualizations for dashboards
ML.NET	Disease prediction via structured dataset classification models	Microsoft's free, open-source ML library for C#; no Python runtime needed for inference; seamless .NET integration
ONNX Runtime (C#)	Medical image classification using pre-trained CNN models	Free, open-source inference engine; runs Python-trained deep learning models in C#; supports ResNet, DenseNet architectures
Ollama (LLaMA 3)	AI chatbot engine for multi-modal clinical reasoning and text analysis	Free, self-hosted LLM inference server; runs LLaMA 3 locally with no API costs; context-aware clinical insights; complete data privacy

Technology	Purpose	Rationale
Whisper.cpp	Voice-to-text transcription for doctor voice notes and chatbot voice input	Free, open-source speech recognition; C++ implementation for fast CPU/GPU inference; medical term correction pipeline
NLP Custom Pipeline (ML.NET)	Clinical note summarization and medical entity extraction	Free ML.NET-based pipeline; extracts symptoms, medications, procedures from free-text notes; F1 score exceeding 0.80 on clinical data
SignalR	Real-time bidirectional chat and alert notifications	Built into ASP.NET Core at no cost; supports WebSocket, Server-Sent Events, and long polling fallback
PostgreSQL 16 + EF Core	Primary relational database with ORM for data persistence	Free, open-source enterprise-grade RDBMS; EF Core provides type-safe LINQ queries and automatic migrations; pgvector extension for vector embeddings
pgvector Extension	Vector similarity search for RAG-enhanced chatbot context retrieval	Free PostgreSQL extension; stores and queries LLM embedding vectors for context-aware chatbot responses
Redis / In-Memory Cache	Session management and performance optimization	Free, open-source in-memory data store; sub-millisecond caching for frequently accessed data; reduces database load
MinIO Object Storage	Medical image and document file storage	Free, open-source S3-compatible object storage; runs self-hosted with no cloud dependency; replaces Azure Blob / AWS S3
ASP.NET Core Identity + JWT	Secure authentication with role-based claims	Free, built-in ASP.NET Core authentication framework; short-lived access tokens with refresh token rotation for both desktop and web clients
QuestPDF	Professional PDF report generation and download	Free, open-source (MIT license) .NET library for programmatic PDF creation; replaces iTextSharp which has restrictive AGPL licensing
Serilog + Grafana Loki	Structured logging with centralized aggregation and alerting	Free, open-source logging pipeline; Loki for log aggregation; Grafana for dashboards and alerts; replaces Seq which is commercial
Docker + Docker Compose	Containerization and orchestration of all services	Free, open-source container platform; one-command deployment; network isolation; consistent environments across dev/test/prod
Nginx	Reverse proxy, SSL termination, and load balancing	Free, open-source web server; handles HTTPS termination and request routing to API and MVC containers
Hangfire	Background job scheduling for risk recalculation and model retraining	Free, open-source background job framework for .NET; persistent job queue in PostgreSQL; dashboard for monitoring
FluentValidation	Input validation rules for API request models	Free, open-source validation library; type-safe validation rules; integrates with ASP.NET Core model binding
MediatR	Mediator pattern for decoupled request/response handling	Free, open-source mediator implementation; reduces coupling between controllers and business logic
Git + GitHub	Version control and collaborative development	Free distributed version control; GitHub provides free private repositories for academic projects

Table 5: MAPS Technology Stack — All Free/Open-Source

4. Module A: Admin Panel

The Admin Panel is the command center of the MAPS platform, providing administrators with comprehensive oversight and control over every aspect of the system. As the highest tier in the role hierarchy, administrators have unrestricted access to user management, system analytics, AI model configuration, audit trails, Docker deployment management, and security settings. The Admin Panel is organized into eight distinct sub-sections, each addressing a critical administrative function.

4.1 User Lifecycle Management

The User Lifecycle Management module provides administrators with complete control over user accounts from creation through deactivation. This includes the ability to create new doctor and patient accounts, assign roles and permissions, manage account activation and deactivation, and handle password resets. The module supports bulk operations for efficient management of large user bases, including CSV import for batch user creation and bulk status updates. Every user lifecycle event is logged in the audit trail, ensuring complete accountability for administrative actions. The registration process incorporates a Registration Lock mechanism that requires admin approval before new accounts become active, preventing unauthorized self-registration and ensuring that only verified medical professionals and validated patients gain system access.

4.2 Doctor-Patient Assignment Engine

The Doctor-Patient Assignment Engine manages the critical relationship between doctors and their assigned patients. Administrators can create, modify, and terminate assignments, transfer patients between doctors, and view assignment histories. The engine supports both manual assignment through an administrative interface and rule-based auto-assignment that considers factors such as doctor specialization, current patient load, and department availability. When a patient is transferred between doctors, all associated medical records, prescriptions, and AI prediction results are automatically linked to the new doctor while preserving the complete historical record under the previous doctor for audit purposes. The assignment engine also manages chat permissions, ensuring that doctors can only communicate with their assigned patients through the SignalR chat system.

4.3 System Analytics Dashboard

The System Analytics Dashboard provides administrators with real-time visibility into platform performance, usage patterns, and operational metrics. The dashboard features interactive Chart.js visualizations showing prediction volumes by disease type, accuracy trends over time, doctor workload distribution, patient registration rates, and system resource utilization. Docker container health status is displayed with real-time monitoring through Grafana integration, showing CPU, memory, and network metrics for each containerized service. The dashboard supports date range filtering, department-level drill-down, and export to PDF reports using QuestPDF.

4.4 AI Model Administration

The AI Model Administration module gives administrators control over the machine learning models powering the platform. This includes viewing currently deployed models, monitoring performance metrics, updating model versions, and configuring prediction thresholds. Administrators can upload new ONNX model files, set confidence thresholds for different prediction types, and manage feature importance weights. Model versioning allows rollback to previous versions if a newly deployed model exhibits degraded performance. The module also manages the Ollama LLM configuration for the AI chatbot, including model selection, temperature settings, and maximum token limits.

4.5 Feedback Analytics

The Feedback Analytics module aggregates and analyzes patient feedback submitted through the Patient Module. Administrators can view overall satisfaction scores, sentiment analysis results, and detailed feedback comments. The module provides trend analysis showing how satisfaction metrics change over time, correlation analysis between

feedback scores and specific doctors or departments, and automated alerting when feedback patterns indicate potential quality of care issues. Feedback data is processed using NLP sentiment analysis to automatically categorize comments as positive, negative, or neutral, with key theme extraction identifying recurring issues.

4.6 Audit Trail and Compliance

The Audit Trail and Compliance module provides a comprehensive, tamper-proof record of every action performed within the MAPS platform. Every user login, data access event, prediction request, prescription creation, chatbot interaction, administrative action, and configuration change is logged with full context including user identity, timestamp, IP address, action type, and affected entities. The audit trail is append-only, meaning records cannot be modified or deleted even by administrators, ensuring the integrity of the compliance trail. Automated alerts are triggered for suspicious activity patterns including multiple failed login attempts, bulk data access, and off-hours administrative actions.

4.7 System Configuration and Security

The System Configuration and Security module provides administrators with centralized control over platform settings including password complexity requirements, session timeout durations, JWT token expiration periods, CORS allowed origins, and rate limiting thresholds. Docker Compose configuration management allows administrators to view and update container settings, restart services, and monitor health check status through an integrated interface. The security module also manages the Registration Lock toggle, two-factor authentication settings, and IP whitelisting for clinic-based desktop clients.

4.8 Admin Dashboard Layout

The Admin Dashboard is designed as a single-page application with a persistent sidebar navigation and a dynamic content area. The sidebar provides quick access to all eight administrative sub-sections, with visual indicators for pending actions such as registration approvals, unread alerts, and compliance review items. The main content area features a customizable widget-based layout where administrators can arrange analytics cards, quick-action buttons, Docker container status, and notification feeds. The dashboard is fully responsive, supporting desktop, tablet, and mobile form factors.

5. Module B: Doctor Module (Enhanced)

The Doctor Module is the most feature-rich and technically sophisticated component of the MAPS platform, comprising sixteen distinct features spanning AI-powered diagnostics, a personalized multi-modal AI chatbot, real-time communication, clinical documentation, voice dictation, drug interaction checking, medical literature search, and intelligent scheduling. Seven AI-driven features transform this module into a comprehensive AI-augmented clinical decision support system. The flagship addition is the AI Personalized Chatbot, a doctor-only feature that accepts text, image, voice, and document inputs to provide context-aware clinical suggestions powered by a locally-hosted Ollama LLaMA 3 model with RAG-enhanced context retrieval.

5.1 AI-Powered Clinical Workflow

The AI-powered clinical workflow within the Doctor Module creates a continuous loop from patient consultation through AI analysis, clinical review, chatbot-assisted research, and patient feedback. The seven AI features interconnect to form a seamless pipeline: the AI Disease Prediction Engine processes symptoms and returns probability-ranked diagnoses; the Differential Diagnosis feature extends this to multi-condition analysis; the Risk Assessment module continuously monitors patient health data; the Medical Image Analysis pipeline processes uploaded scans; Clinical Notes with AI Summarization automatically structures consultation documentation; the AI Chatbot provides multi-modal clinical reasoning support; and Voice-to-Text Dictation enables hands-free clinical note creation. This workflow ensures that every AI-generated insight passes through the doctor's review before reaching the patient, maintaining clinical accountability.

5.2 Doctor Dashboard

The Doctor Dashboard serves as the central hub for all clinical activities, providing a unified view of the doctor's daily workflow. The dashboard is organized into distinct panels: a patient queue sorted by AI-calculated priority tier with color-coded urgency indicators; an AI Insights panel showing recent prediction results, risk alerts, and chatbot conversation summaries; a schedule overview displaying upcoming appointments with Smart Priority Engine rankings; a quick-action panel for one-click access to prescription writing, clinical notes, and the AI chatbot; and a notification feed aggregating SignalR alerts for new patient assignments, urgent risk escalations, and patient messages. The dashboard integrates real-time SignalR connections, automatically updating when new patients are assigned or risk scores change.

5.3 AI Disease Prediction Engine

The AI Disease Prediction Engine is the core diagnostic tool within the Doctor Module, processing patient symptoms through ML.NET classification models to return probability-ranked disease predictions. The engine uses ML.NET models trained on clinical datasets to map symptom combinations to disease probabilities. Doctors interact with the prediction engine through a symptom selection interface where they can search for and select symptoms from a standardized medical vocabulary, specify symptom duration and severity, and add relevant patient history context. The engine processes these inputs through the ML.NET prediction pipeline and returns results within seconds, displayed through an intuitive interface that clearly separates the AI suggestion from the doctor's own clinical judgment. Disease prediction operates across three input modalities: structured dataset input, image input, and text prompt input processed by Ollama.

Disease	Dataset Input	Image Input	Text Prompt
Diabetes	Glucose, BMI, Age	-	Yes
Pneumonia	-	Chest X-Ray	Yes
Brain Tumour	-	MRI Scan	Yes
Skin Cancer	-	Skin Lesion Photo	Yes
Heart Disease	ECG, Cholesterol	-	Yes

Table 6: Supported Diseases and Input Modalities

5.4 AI-Assisted Differential Diagnosis

The AI-Assisted Differential Diagnosis feature extends the basic prediction engine by generating a ranked list of possible diagnoses rather than a single prediction. When a doctor inputs a patient's symptoms, the system returns multiple potential conditions ranked by probability, along with reasoning paths that explain why each condition is being suggested, and recommended confirmatory or rule-out tests for each candidate. This mirrors the clinical practice of differential diagnosis, where physicians consider multiple possible explanations before arriving at a final diagnosis. The technical implementation uses a multi-output classification model that generates probability distributions across the disease catalog, then filters and ranks the top candidates based on clinical relevance thresholds. Clinical filtering rules remove statistically possible but clinically implausible diagnoses based on patient demographics and symptom patterns, ensuring that the differential list contains only clinically actionable candidates. Each differential diagnosis entry includes the probability score, reasoning chain linking specific symptoms to the diagnosis, and recommended confirmatory tests mapped from clinical practice guidelines.

5.5 Patient Risk Assessment and Proactive Alerts

The Patient Risk Assessment module operates as a continuous background process that recalculates patient risk scores whenever new clinical data enters the system. The risk scoring engine aggregates features from multiple sources including recent AI prediction outcomes, vital sign trends from clinical notes, appointment attendance patterns, prescription adherence indicators, and the patient's overall health trajectory. A gradient boosting model processes these features to produce a risk score between 0 and 100, which is then classified into the four-tier urgency system. The risk

assessment results drive the Smart Appointment Priority Engine and generate proactive SignalR alerts to doctors when a patient's risk score crosses a tier boundary, ensuring that deteriorating patients receive timely clinical attention without requiring the doctor to manually monitor all patient records.

Urgency Tier	Risk Score Range	Alert Response Time	Clinical Action
Emergency	Above 80	Same-day alert	Immediate clinical intervention required
Urgent	60 - 80	Within 24 hours	Priority consultation and further testing
Normal	30 - 60	Within 3 days	Routine follow-up scheduling
Follow-up	Below 30	Within 1 week	Standard monitoring and wellness check

Table 7: Four-Tier Risk Assessment Urgency Classification

5.6 Clinical Notes with AI Summarization

The Clinical Notes module allows doctors to document consultations using free-text notes, which are then processed by an NLP pipeline that automatically summarizes the content, extracts key medical entities, and structures the information into the patient's health record. The NLP engine identifies and categorizes entities including symptoms with onset and duration, vital signs, medications with dosage, procedures, tests ordered, and follow-up instructions. The summarization engine generates a concise three-to-five sentence summary capturing key findings, decisions, and action items. Extracted entities are linked to the patient's structured health record, making the data available for AI prediction, risk assessment, and reporting. The NLP pipeline achieves entity extraction F1 scores above 0.80 on clinical documentation. Doctors can also use the Voice-to-Text feature (Section 5.14) to dictate notes hands-free.

5.7 Medical Image Analysis

The Medical Image Analysis feature enables doctors to upload medical images for AI-powered analysis using CNN models served through ONNX Runtime. The pipeline includes image preprocessing (resizing, normalization, contrast enhancement using CLAHE), CNN inference for feature extraction and classification, post-processing with non-maximum suppression, and report generation with bounding boxes and confidence scores. The doctor reviews the AI analysis, confirms or modifies findings, and adds clinical context before saving to the patient record. Supported imaging modalities include chest X-rays for pneumonia and cardiomegaly detection, skin lesion images for melanoma classification, and MRI scans for brain tumour identification. Image files are stored in MinIO object storage with AES-256 encryption at rest.

5.8 Smart Appointment Priority Engine

The Smart Appointment Priority Engine integrates with the Risk Assessment module to automatically prioritize appointment scheduling based on patient urgency. When a patient books an appointment, the engine retrieves their current risk score and assigns a priority tier: Emergency (above 80) triggers same-day scheduling, Urgent (60-80) within 24 hours, Normal (30-60) within three business days, and Follow-up (below 30) within one week. The engine performs conflict detection to prevent double-booking, manages doctor availability windows, and considers appointment type duration. When a patient books an appointment, the system automatically calculates their risk score and assigns the appropriate priority tier. Administrators can override priority levels when clinical circumstances warrant deviation from AI-recommended scheduling.

5.9 Patient Management

The Patient Management interface provides doctors with a comprehensive view of all their assigned patients, including demographic information, medical history, current prescriptions, recent AI predictions, risk assessment scores, and chatbot interaction summaries. The patient list supports filtering by risk level, condition type, and appointment status.

Each patient's detail view includes a timeline of all clinical interactions, AI prediction results, medication changes, chatbot-assisted research queries, and appointment history, providing a complete longitudinal health record. The patient profile also displays the doctor's chatbot conversation history related to that patient, creating a seamless link between AI-assisted research and clinical decision-making.

5.10 Prescription Management

The Prescription Management module allows doctors to create, modify, and manage prescriptions for assigned patients. The interface supports a structured prescription form including medication selection from a drug database, dosage and frequency specification, duration of treatment, and special instructions. The module includes an AI-powered drug interaction checker (detailed in Section 5.15) that flags potential conflicts between newly prescribed medications and existing prescriptions, and provides warnings for known allergies documented in the patient's record. Prescriptions are stored in the PostgreSQL database with full audit trails tracking every modification.

5.11 Doctor-Patient Chat System

The Doctor-Patient Chat System provides real-time communication through SignalR WebSocket connections. The interface supports text messages, image sharing for wound photos or test results, and file attachments including PDF and DOCX documents. The chat system enforces the role-based permission matrix, ensuring that doctors can only chat with assigned patients and vice versa. Chat messages are persisted in PostgreSQL with timestamps and delivery status, providing a complete communication history. The chat interface includes typing indicators, read receipts, and the ability to share AI prediction results directly within the conversation, allowing doctors to contextualize diagnostic findings for patients during real-time consultations.

5.12 Medical Reports Generation

The Medical Reports Generation module allows doctors to create professional PDF reports for patients using QuestPDF, a free and open-source .NET library that replaces the restrictive AGPL-licensed iTextSharp. Doctors can generate individual patient health summaries, AI prediction result reports with confidence scores and visualizations, prescription summaries, referral letters, and comprehensive consultation reports. The report builder provides customizable templates with the ability to include or exclude sections such as AI prediction details, risk assessment history, medication lists, and clinical notes. Generated reports are stored in MinIO and can be downloaded or shared with patients through the chat system, providing professional medical report generation and download capability.

5.13 AI Personalized Chatbot for Doctors

FLAGSHIP FEATURE — Doctor-Only Multi-Modal AI Assistant

The AI Personalized Chatbot is the flagship feature of the enhanced MAPS Doctor Module, providing doctors with an intelligent clinical assistant that accepts four distinct input modalities: text queries, medical images, voice recordings, and document uploads. Unlike general-purpose chatbots, the MAPS chatbot is deeply integrated with the patient health record system and uses Retrieval-Augmented Generation (RAG) to provide context-aware clinical suggestions grounded in the patient's actual medical history, current medications, and previous AI prediction results. The chatbot is powered by Ollama hosting LLaMA 3, a free and open-source large language model that runs entirely within the Docker container infrastructure, ensuring complete data privacy with no external API calls.

The chatbot is strictly limited to the Doctor role and is not accessible to patients or administrators. This design decision ensures that AI-generated clinical suggestions are always mediated by a qualified medical professional who can contextualize, validate, or reject the chatbot's output. The chatbot operates under a safety-first protocol that includes mandatory disclaimer injection on every response, prohibition of direct prescription recommendations, and clear visual separation between AI suggestions and clinical decisions. Every chatbot interaction is logged in the audit trail and linked to the patient context for complete accountability.

5.13.1 Multi-Modal Input System

The chatbot accepts four input types, each processed through a dedicated pre-processing pipeline before reaching the Ollama LLM engine for response generation:

Input Type	Processing Pipeline	Example Use Cases	Technology
Text Input	Tokenization, medical NER, symptom extraction, query classification	Ask about symptoms, drug dosages, clinical guidelines, differential diagnosis support	Ollama LLaMA 3 + NLP Pipeline
Image Input	Image upload, ONNX CNN inference, feature extraction, visual description generation	Upload X-ray/MRI/skin lesion for AI second opinion, compare with previous scans	ONNX Runtime + Ollama Vision
Voice Input	Audio capture, Whisper.cpp transcription, medical term correction, text routing	Dictate clinical observations, ask questions hands-free during procedures	Whisper.cpp + Ollama
Document Input	PDF/DOCX parsing, text extraction, structure recognition, content summarization	Upload lab reports, discharge summaries, referral letters for AI analysis	iTextSharp/DocX + Ollama

Table 8: Chatbot Multi-Modal Input Processing

5.13.2 RAG-Enhanced Context Retrieval

The chatbot uses Retrieval-Augmented Generation (RAG) to ground its responses in the patient's actual medical data rather than relying solely on the LLM's training knowledge. When a doctor initiates a chatbot session for a specific patient, the RAG system performs the following steps: first, it retrieves the patient's health records, recent AI predictions, current prescriptions, risk assessment history, and clinical notes from PostgreSQL; second, it converts this data into vector embeddings using the pgvector extension and performs similarity search against the doctor's query; third, it constructs a context prompt that includes the most relevant patient data along with the doctor's query; and fourth, it sends this enriched prompt to Ollama for response generation. This approach ensures that chatbot responses are personalized to the specific patient and grounded in factual medical data, significantly reducing hallucinations and improving clinical relevance.

5.13.3 Conversation Memory and Personalization

The chatbot maintains conversation memory across sessions, allowing doctors to reference previous queries and build upon earlier investigations. Conversation history is stored in the ChatbotMemory table with vector embeddings for semantic retrieval. When a doctor starts a new session, the system retrieves relevant previous conversations using vector similarity search, providing continuity of thought across multiple clinical encounters. The personalization layer also learns doctor preferences over time, such as preferred diagnostic approaches, commonly referenced clinical guidelines, and frequently asked question patterns, tailoring response style and depth to each individual doctor's workflow.

5.13.4 Safety Guardrails

The chatbot implements multiple safety mechanisms to ensure responsible AI-assisted clinical support. Every response includes a mandatory disclaimer stating that the AI suggestion is for clinical decision support only and must be validated by the doctor. The Safety Guard module prevents the chatbot from generating direct prescription recommendations, instead providing medication information with explicit instructions to consult pharmacological references. Responses that reference specific diagnoses or treatment plans include confidence indicators and source citations from the RAG context. The chatbot declines to provide emergency medical advice and instead redirects doctors to established emergency protocols. All chatbot interactions are logged in the audit trail with full context, ensuring complete traceability and accountability for AI-assisted clinical decisions.

5.14 Voice-to-Text Clinical Dictation

The Voice-to-Text Clinical Dictation feature enables doctors to create clinical notes, prescriptions, and chatbot queries using voice input, providing a hands-free interaction mode that is particularly valuable during patient examinations when typing is impractical. The feature uses Whisper.cpp, a free and open-source speech recognition engine that runs within

the Docker infrastructure, ensuring that voice data never leaves the local network. The voice processing pipeline captures audio through the device microphone, streams the audio to the Whisper.cpp container for transcription, applies medical term correction using a custom dictionary that maps common speech recognition errors to correct medical terminology, and routes the transcribed text to the appropriate destination (clinical notes, chatbot, or prescription form). The system supports continuous dictation with automatic sentence segmentation and punctuation restoration, achieving transcription accuracy above 90% for medical vocabulary after correction.

5.15 Drug Interaction AI Checker

The Drug Interaction AI Checker extends the Prescription Management module with intelligent drug-drug and drug-allergy interaction detection. When a doctor prescribes a new medication, the system automatically checks it against the patient's current prescriptions, known allergies, and medical conditions. The interaction checker uses a locally-maintained drug interaction database stored in PostgreSQL, supplemented by Ollama LLM-powered analysis for complex interaction scenarios that go beyond the structured database. The system classifies interactions into three severity levels: contraindicated (red alert, prescription blocked until doctor override), major interaction (orange warning, doctor must acknowledge before proceeding), and minor interaction (yellow advisory, informational only). The AI checker also considers patient-specific factors such as age, weight, kidney function, and liver function when assessing interaction severity, providing personalized risk assessments rather than generic drug interaction warnings.

5.16 Medical Literature Search (RAG)

The Medical Literature Search feature provides doctors with AI-powered access to medical knowledge bases and clinical guidelines directly from the Doctor Module interface. Using the same RAG architecture that powers the chatbot, the literature search indexes clinical guidelines, drug monographs, and medical reference documents into the pgvector database as vector embeddings. When a doctor searches for information, the system performs semantic similarity search against the indexed literature and returns the most relevant passages along with source citations. The Ollama LLM then synthesizes these retrieved passages into a coherent, contextualized response that directly addresses the doctor's query. This feature eliminates the need for doctors to switch to external browsers during consultations, keeping all clinical research within the MAPS ecosystem where it can be linked to patient records and cited in clinical notes.

6. Module C: Patient Module

The Patient Module provides patients with a secure, intuitive interface for managing their healthcare interactions. Designed with controlled access, the module ensures that patients can view personal health information, communicate with their assigned doctor, and manage appointments, while all AI-generated insights are routed through their doctor to maintain clinical oversight. The Patient Module does not include an AI chatbot feature; all AI predictions, risk assessments, and diagnostic suggestions are shared by the doctor during consultations or through the doctor-mediated results viewing system. This design ensures that AI serves as a clinical support tool rather than a self-service diagnostic mechanism.

6.1 Patient Dashboard

The Patient Dashboard provides a welcoming, easy-to-navigate interface displaying upcoming appointments, recent health updates, and messages from the assigned doctor. The dashboard features appointment cards with date, time, and priority indicators, a health summary panel showing key vitals and active prescriptions, a messaging widget for quick communication, and a notification feed for system announcements and doctor-shared results. The interface is designed with accessibility in mind, featuring large touch targets, high-contrast color schemes, and clear typography.

6.2 Appointment Booking

Patients can book, reschedule, and cancel appointments through an intuitive scheduling interface. When a patient initiates a booking, the Smart Appointment Priority Engine automatically calculates their priority tier based on their current risk score and presents available slots accordingly. High-risk patients are offered earlier time slots, while routine

follow-up patients are shown standard availability. The booking system prevents double-booking and manages doctor availability in real-time through SignalR synchronization.

6.3 Health Records and History

Patients can view their complete health record including demographic information, medical history, past prescriptions, appointment history, and doctor-shared AI prediction results. Health records are presented in a timeline view that organizes clinical events chronologically, making it easy for patients to track their health journey. Records are filtered through the role-based permission system to show only the patient's own data, with no access to other patients' information.

6.4 View AI Predictions (Doctor-Shared Only)

Patients can view AI prediction results only when their assigned doctor chooses to share them. The sharing mechanism allows doctors to select specific predictions, add their own interpretation and recommended next steps, and then make the results visible to the patient through the Patient Module. The shared prediction view includes the primary condition identified, the confidence percentage in a patient-friendly format, the doctor's interpretation and recommended next steps, and any lifestyle modifications or follow-up tests recommended. This mediated sharing approach ensures that patients receive AI insights within the appropriate clinical context.

6.5 Feedback System

The Feedback System allows patients to rate their consultation experience and provide detailed feedback on the care they received. The feedback form includes a star rating scale, structured evaluation categories (communication quality, wait time, overall experience), and a free-text comment field. Feedback submissions are anonymous to the assigned doctor by default. All feedback data is aggregated and analyzed through the Admin Module's Feedback Analytics dashboard, providing a compulsory post-consultation feedback mechanism that delivers reliable, structured doctor performance data to administrators.

7. AI/ML Pipeline

The MAPS AI/ML pipeline encompasses four distinct processing streams that serve different clinical functions: the Disease Prediction Pipeline, the Differential Diagnosis Pipeline, the Risk Assessment Pipeline, and the Medical Image Analysis Pipeline. Each pipeline is independently deployable and scalable, allowing the system to allocate computational resources based on demand patterns without affecting other AI features. A feedback loop ensures that prediction results and risk scores feed back into the data pool, improving future model accuracy through periodic retraining. All models are served within the Docker container infrastructure, with ONNX models stored as shared volumes and ML.NET models embedded in the API container.

7.1 AI/ML Pipeline Diagram

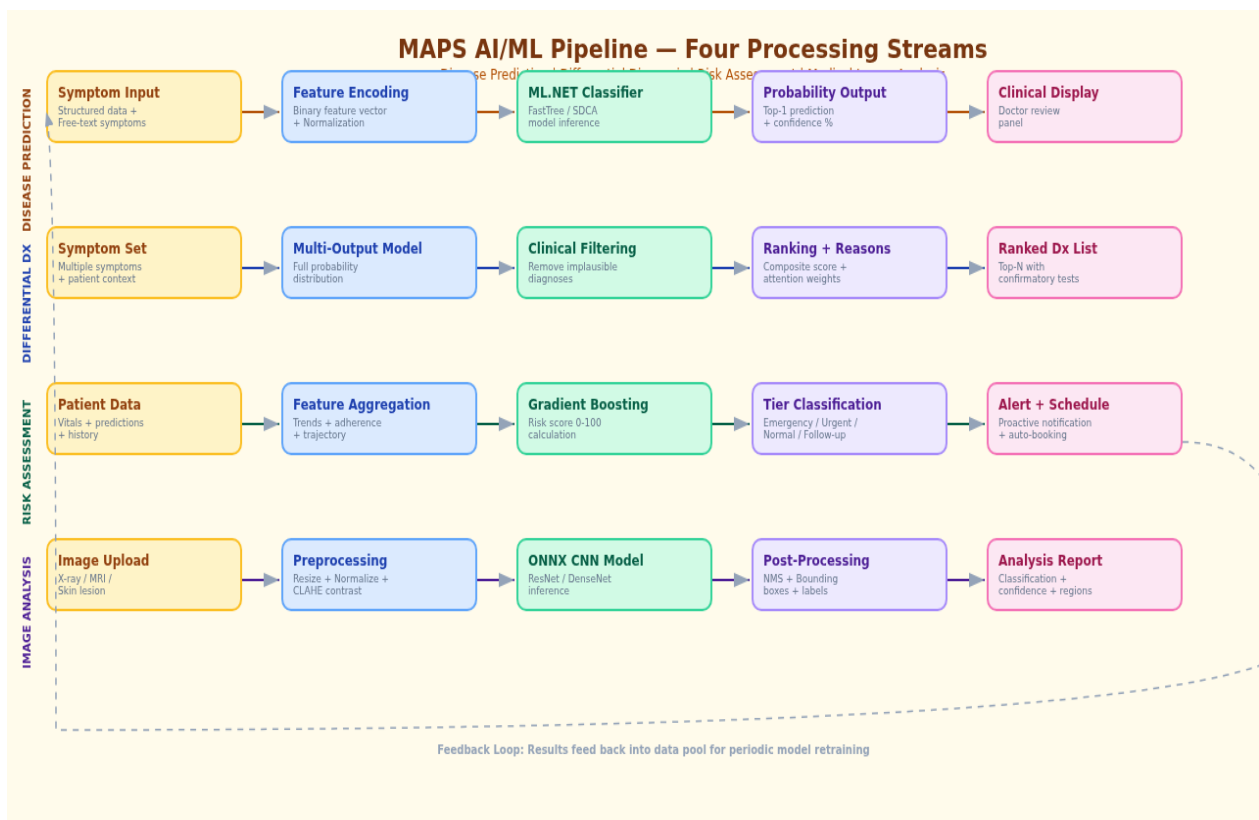


Figure 2: MAPS AI/ML Pipeline — Four Processing Streams with Feedback Loop

7.2 Disease Prediction Pipeline

The Disease Prediction Pipeline processes symptom inputs through a multi-stage workflow: symptom encoding converts the selected symptoms into a binary feature vector, the ML.NET prediction model applies the trained classification algorithm to generate probability distributions across the disease catalog, and the result formatting stage selects the top prediction with confidence percentage and displays it to the doctor for clinical review. The pipeline supports three input modalities: structured data, image input through the ONNX image pipeline, and text prompt analysis through the Ollama LLM.

7.3 Differential Diagnosis Pipeline

The Differential Diagnosis Pipeline extends the base prediction pipeline by retaining the full probability distribution rather than selecting only the top result. Clinical filtering rules remove statistically possible but clinically implausible diagnoses based on patient demographics and symptom patterns, the remaining candidates are ranked by a composite score balancing model probability with clinical relevance, and reasoning chains are generated for each candidate by extracting attention weights that link specific symptoms to the diagnosis. Each differential diagnosis entry includes recommended confirmatory tests mapped from clinical practice guidelines.

7.4 Risk Assessment Pipeline

The Risk Assessment Pipeline operates as a background process that recalculates patient risk scores whenever new clinical data enters the system. The pipeline aggregates features from multiple sources including recent AI prediction outcomes, vital sign trends from clinical notes, appointment attendance patterns, prescription adherence indicators, and the patient's overall health trajectory. A gradient boosting model processes these features to produce a risk score between 0 and 100, which is then classified into the four-tier urgency system and compared against the previous score to determine trend direction.

7.5 Medical Image Analysis Pipeline

The Medical Image Analysis Pipeline handles uploaded medical images through a computer vision workflow: image preprocessing (resizing to model input dimensions, pixel normalization, and contrast enhancement using CLAHE), inference through a CNN model that outputs classification probabilities and bounding box coordinates, post-processing that applies non-maximum suppression to eliminate duplicate detections, and report generation producing a structured analysis summary including detected regions, classification labels, and confidence scores. The pipeline supports multiple model architectures for different imaging modalities, with model selection handled automatically based on the uploaded image type.

8. AI Chatbot Architecture

The AI Chatbot Architecture section provides a detailed technical description of the multi-modal chatbot system that powers the Doctor Module's personalized AI assistant. The chatbot is built on a pipeline architecture that processes four distinct input types through specialized pre-processing modules, enriches queries with RAG-retrieved patient context, generates responses through a locally-hosted Ollama LLaMA 3 model, and applies safety guardrails before delivering the response to the doctor. The entire chatbot system runs within Docker containers, ensuring isolation from other platform services and complete data privacy.

8.1 Chatbot Workflow Diagram

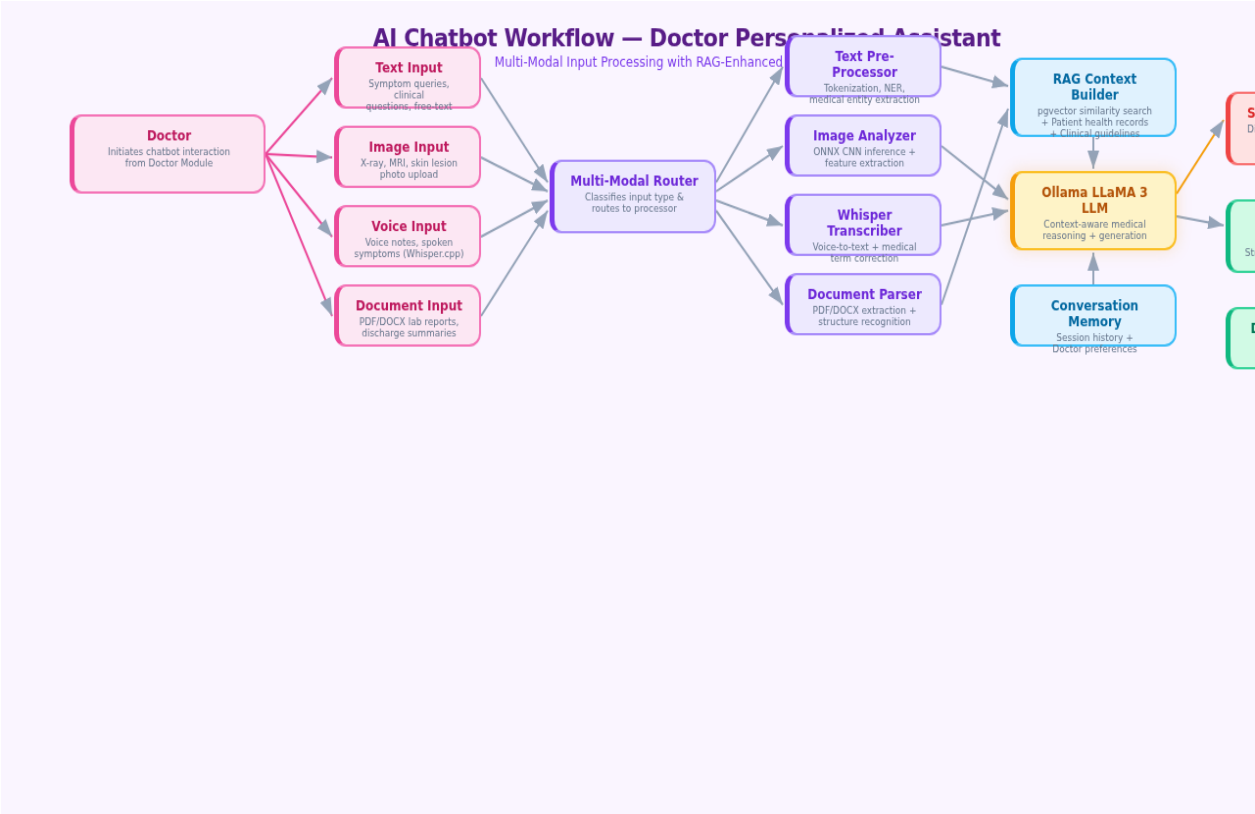


Figure 3: AI Chatbot Workflow — Multi-Modal Input Processing with RAG-Enhanced Clinical Context

8.2 Multi-Modal Input Router

The Multi-Modal Input Router is the entry point for all chatbot interactions. When a doctor submits a query, the router first classifies the input type based on the MIME type and content structure: text inputs are routed directly to the NLP pre-processor, image inputs are sent to the ONNX image analyzer, voice inputs are forwarded to the Whisper.cpp transcription service, and document inputs are parsed by the document extraction module. The router maintains a conversation context object that accumulates all inputs and outputs within the current session, enabling multi-turn conversations where doctors can combine different input modalities. For example, a doctor might first upload a chest X-ray image, receive the AI analysis, and then ask a follow-up text question about the findings, with the chatbot

maintaining full context of both the image analysis and the text conversation.

8.3 RAG Context Builder

The RAG Context Builder assembles the contextual information that accompanies each chatbot query to the Ollama LLM. The builder operates in three stages: data retrieval, embedding generation, and context construction. In the data retrieval stage, the system queries PostgreSQL for the patient's health records, recent AI predictions, current prescriptions, risk assessment scores, and clinical notes. In the embedding generation stage, the Ollama embedding model converts both the doctor's query and the retrieved patient data into vector representations stored and searched using the pgvector PostgreSQL extension. In the context construction stage, the most relevant patient data fragments (ranked by vector similarity to the query) are assembled into a structured context prompt that includes the patient summary, relevant medical history, and the current query. This context is prepended to the doctor's query before submission to the Ollama LLM, ensuring that responses are grounded in factual patient data.

8.4 Ollama LLM Integration

Ollama serves as the local LLM inference engine for the chatbot, hosting the LLaMA 3 model within a dedicated Docker container. The choice of Ollama over cloud-based LLM APIs (such as OpenAI or Groq) is driven by three critical requirements: data privacy (patient data never leaves the local network), cost elimination (no per-token API charges), and availability (the system works offline without internet connectivity). The Ollama container is configured with temperature 0.3 for clinical precision, maximum token limit of 2048 per response, and system prompts that establish the medical assistant persona with safety guidelines. The API communication between the MAPS API container and the Ollama container occurs over the internal Docker network, ensuring low latency and secure communication without exposure to the external network.

8.5 Safety Guard Module

The Safety Guard Module is the final checkpoint before chatbot responses reach the doctor. Every response generated by Ollama passes through the safety guard which performs the following checks: disclaimer injection adds a mandatory clinical decision support disclaimer to every response; prescription filter blocks any direct medication recommendation and replaces it with a prompt to consult pharmacological references; emergency redirect detects queries about emergency conditions and provides established emergency protocol references instead of AI-generated advice; and confidence tagging adds confidence indicators based on the RAG context relevance score, flagging responses with low context relevance for additional clinical scrutiny. The safety guard also enforces content policies, preventing the chatbot from generating responses outside the clinical domain.

9. Database Design

The MAPS database is implemented in PostgreSQL 16, a free and open-source relational database management system that replaces the proprietary SQL Server used in the original design. PostgreSQL provides enterprise-grade features including Transparent Data Encryption, JSONB support for flexible data structures, full-text search, and the pgvector extension for vector similarity search used by the RAG chatbot system. The database schema consists of seventeen tables, including three new tables added for the AI chatbot feature: ChatSessions, ChatbotMessages, and ChatbotMemory.

9.1 ER Diagram

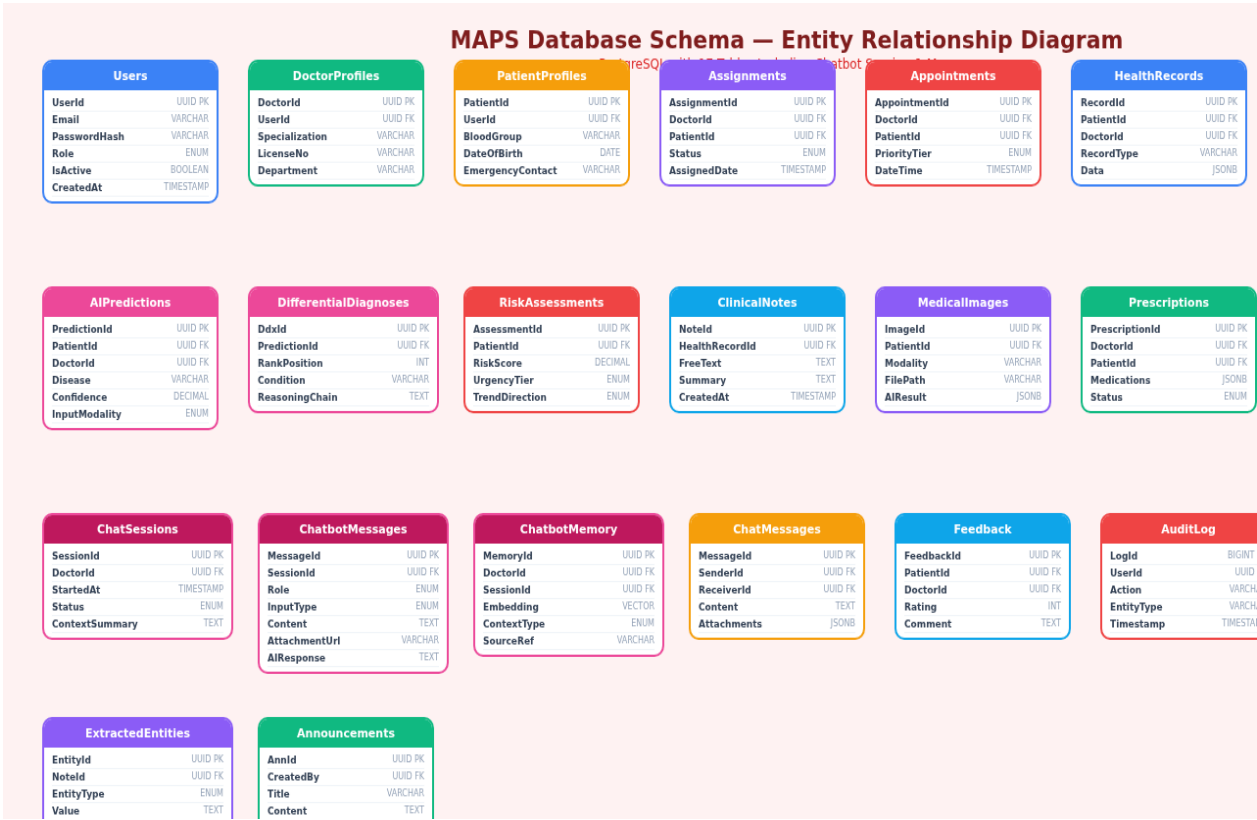


Figure 4: MAPS Database Schema — Entity Relationship Diagram (17 Tables)

9.2 Table Definitions

Table	Purpose	Key Relationships
Users	All user accounts with role assignments	DoctorProfiles, PatientProfiles
DoctorProfiles	Doctor-specific data (specialization, license)	Users, Assignments, Prescriptions
PatientProfiles	Patient-specific data (blood group, DOB)	Users, Assignments, HealthRecords
Assignments	Doctor-Patient relationship mapping	Doctors, Patients, AuditLog
Appointments	Scheduling data with priority and status	Doctors, Patients, RiskAssessments
HealthRecords	Longitudinal clinical data entries	Patients, Doctors, ClinicalNotes
AIPredictions	Prediction results with confidence scores	Patients, Doctors, DifferentialDiagnoses
DifferentialDiagnoses	Multi-condition ranked diagnosis results	AIPredictions, SuggestedTests
RiskAssessments	Patient risk scores with trend tracking	Patients, Doctors, Appointments
ClinicalNotes	Free-text notes with NLP-extracted entities	HealthRecords, ExtractedEntities
ExtractedEntities	Structured medical entities from NLP	ClinicalNotes, HealthRecords
MedicalImages	Uploaded images with AI analysis results	Patients, Doctors, HealthRecords
Prescriptions	Medication orders with dosage and duration	Doctors, Patients, HealthRecords
ChatMessages	Real-time message persistence	Users (Sender/Receiver)
Feedback	Patient ratings and comments	Patients, Doctors
AuditLog	Tamper-proof action logging	Users (all actions)
Announcements	System-wide and targeted announcements	Admin (creator)
ChatSessions	AI chatbot session tracking	DoctorProfiles, ChatbotMessages

Table	Purpose	Key Relationships
ChatbotMessages	Chatbot conversation messages with input type	ChatSessions, ChatbotMemory
ChatbotMemory	Vector embeddings for RAG context retrieval	DoctorProfiles, ChatSessions

Table 9: MAPS Database Schema — Table Definitions (17 Tables Including Chatbot)

10. Docker Containerization

NEW INFRASTRUCTURE — One-Command Deployment

MAPS is fully containerized using Docker with Docker Compose orchestration, providing consistent deployment across development, testing, and production environments. The containerization strategy separates the platform into four network tiers — Frontend, Backend, AI, and Monitoring — each running in an isolated Docker network with controlled inter-container communication. This approach eliminates the 'it works on my machine' problem, simplifies deployment to a single command ('docker compose up -d'), and provides built-in service isolation and security through Docker network segmentation.

10.1 Docker Deployment Diagram

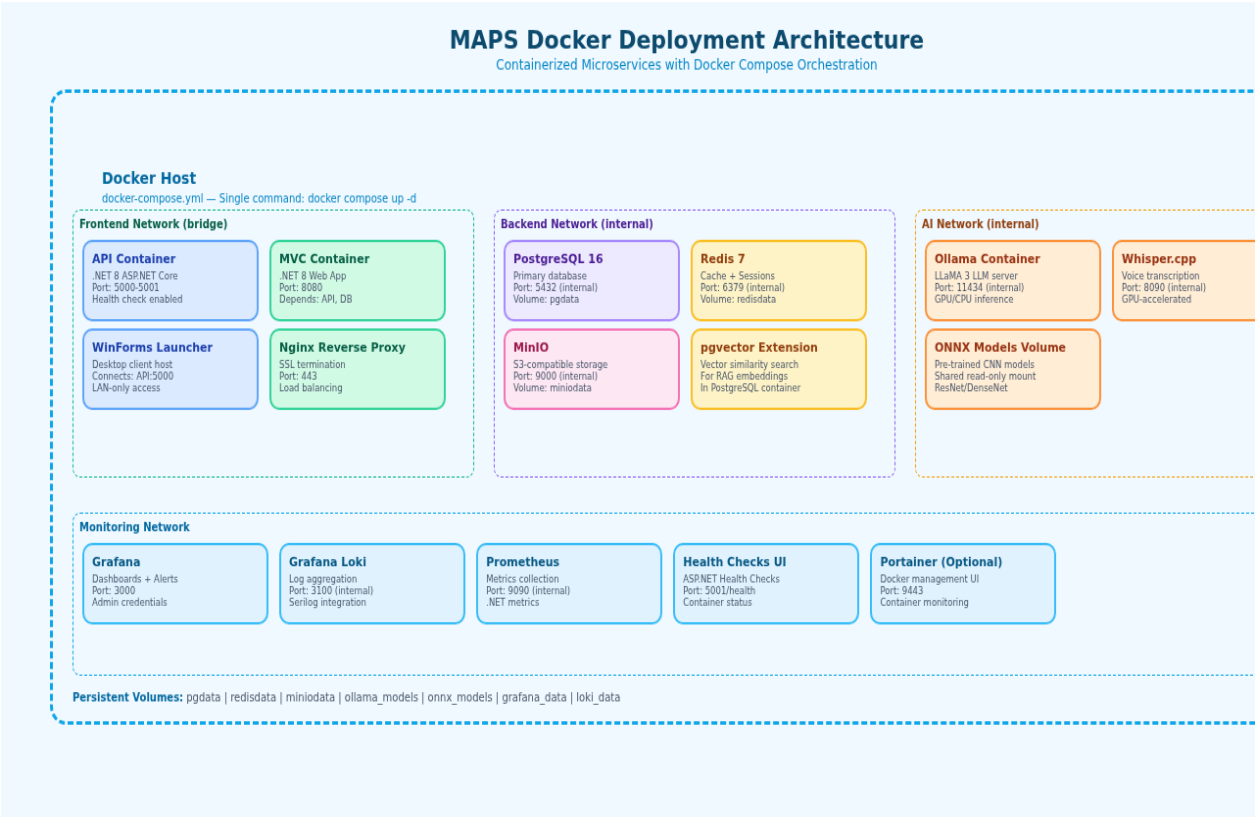


Figure 5: MAPS Docker Deployment Architecture — Containerized Microservices with Docker Compose

10.2 Container Inventory

Container	Image	Port(s)	Network	Volume
API	maps-api:latest (.NET 8)	5000 (internal)	Frontend + Backend	onnx_models (ro)
MVC Web	maps-web:latest (.NET 8)	8080 (internal)	Frontend	-
Nginx	nginx:alpine	443 (external)	Frontend	ssl_certs
PostgreSQL	postgres:16	5432 (internal)	Backend	pgdata

Container	Image	Port(s)	Network	Volume
Redis	redis:7-alpine	6379 (internal)	Backend	redisdata
MinIO	minio/minio	9000 (internal)	Backend	miniodata
Ollama	ollama/ollama	11434 (internal)	AI	ollama_models
Whisper.cpp	maps-whisper:latest	8090 (internal)	AI	-
Grafana	grafana/grafana	3000 (internal)	Monitoring	grafana_data
Loki	grafana/loki	3100 (internal)	Monitoring	loki_data
Prometheus	prom/prometheus	9090 (internal)	Monitoring	-

Table 10: Docker Container Inventory

10.3 Docker Compose Configuration

The docker-compose.yml file defines all services, networks, and volumes required for the MAPS platform. The configuration specifies four custom Docker networks: maps-frontend (bridge mode, exposing Nginx on port 443), maps-backend (internal mode, no external access), maps-ai (internal mode, for Ollama and Whisper communication), and maps-monitoring (internal mode, for logging and metrics). Each container includes health check configurations that Docker Compose uses to manage service dependencies, ensuring that the API container waits for PostgreSQL and Redis to be healthy before starting. Persistent volumes ensure that database data, uploaded files, LLM model weights, and monitoring data survive container restarts and updates. Environment variables are managed through a .env file that separates configuration from code, supporting different settings for development, staging, and production environments.

10.4 Deployment Workflow

The deployment workflow is streamlined to a three-step process: first, clone the repository and configure the .env file with environment-specific settings such as database passwords, JWT signing keys, and Ollama model selection; second, run 'docker compose build' to build the .NET API and MVC containers from the Dockerfile (which uses multi-stage builds for minimal image size); third, run 'docker compose up -d' to start all services in detached mode. Docker Compose automatically handles service startup ordering based on health check dependencies, network creation, and volume initialization. The entire deployment completes within five minutes on a standard server. Updates are performed using 'docker compose pull && docker compose up -d' which performs rolling updates with zero downtime for stateless services. Database migrations are executed automatically by the API container on startup using Entity Framework Core's automatic migration feature.

11. Security Architecture

The MAPS platform implements a multi-layered security architecture that protects patient data, prevents unauthorized access, and ensures regulatory compliance. Security measures span authentication, authorization, data encryption, audit logging, Docker network isolation, and application-level protections, creating a defense-in-depth strategy that addresses threats at every level of the technology stack. The containerized Docker deployment adds an additional security layer through network segmentation, which ensures that sensitive services like PostgreSQL and Ollama are only accessible from authorized containers within the internal Docker network.

11.1 Security Architecture Diagram

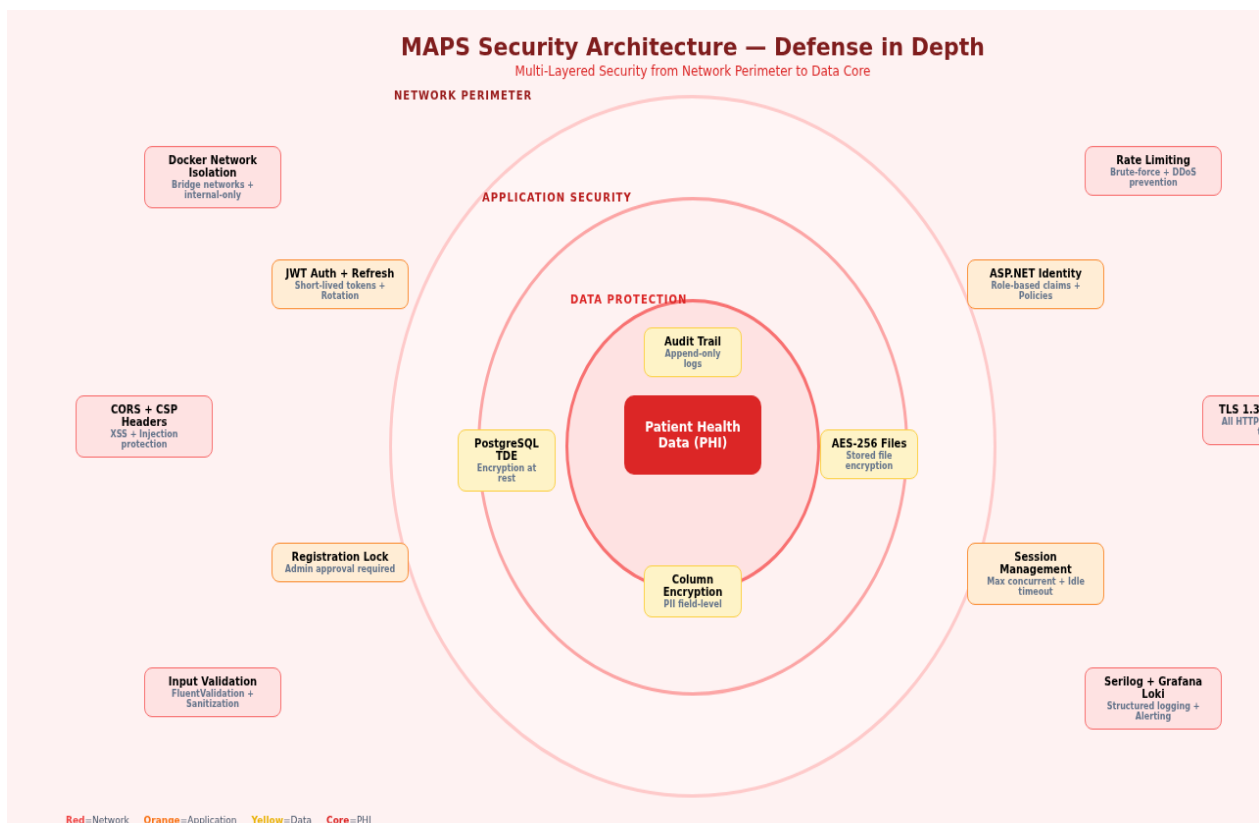


Figure 6: MAPS Security Architecture — Defense in Depth with Docker Network Isolation

11.2 Authentication and Authorization

Authentication is implemented through ASP.NET Core Identity with JWT bearer authentication. Upon login, the system issues a short-lived access token (15-minute expiry) and a refresh token (7-day expiry with rotation). JWT tokens embed the user's role and permission scope, enabling the authorization middleware to make real-time access control decisions without database lookups. Passwords are stored using bcrypt hashing with automatic salting, and the system enforces configurable password complexity requirements. The Registration Lock mechanism requires administrator approval before newly registered accounts become active, preventing unauthorized self-registration. Both the UI and the API enforce this lock, providing double-layer protection.

11.3 Data Encryption

All sensitive data is encrypted both at rest and in transit. TLS 1.3 encrypts all HTTP communications including API requests, WebSocket messages, and file uploads, with SSL termination handled by the Nginx reverse proxy container. Data at rest encryption uses PostgreSQL Transparent Data Encryption for the database and AES-256 encryption for files stored in MinIO. Personally identifiable information fields in the database are additionally encrypted at the column level using Entity Framework Core's value conversion features, providing protection even if the database is directly compromised.

11.4 Network and Application Security

The Docker network architecture provides inherent security through network segmentation. The Backend, AI, and Monitoring networks are configured as internal Docker networks, meaning they have no external internet access and can only be reached by containers on the same network. The Frontend network is the only network with external exposure, limited to the Nginx container on port 443. Application-level security includes rate limiting on all API endpoints, CORS policies restricted to authorized frontend domains, Content Security Policy headers preventing cross-site scripting, and input validation using FluentValidation and parameterized queries to prevent SQL injection. Session management policies control maximum concurrent sessions per user and enforce idle timeouts.

12. API Design

The MAPS platform exposes a RESTful API layer built on ASP.NET Core Web API that serves as the communication backbone between the frontend and backend services. All API endpoints follow consistent naming conventions, use standard HTTP methods for CRUD operations, and return standardized response formats including status codes, error messages, and pagination metadata. API documentation is generated automatically using Swagger/OpenAPI, providing interactive API exploration and testing capabilities.

Endpoint Group	Key Routes	Access Level
Authentication	POST /api/auth/login, /refresh, /register	Public + Admin Approval
User Management	GET/POST/PUT/DELETE /api/users	Admin Only
Doctor-Patient Assignment	GET/POST/PUT/DELETE /api/assignments	Admin Only
AI Predictions	POST /api/predictions, GET /api/predictions/{id}	Doctor / Admin
Differential Dx	POST /api/predictions/differential, GET /api/predictions/{id}/differential	Doctor / Admin
Risk Assessment	GET /api/risks/patient/{id}, GET /api/risks/alerts	Doctor / Admin
Clinical Notes	POST/GET /api/notes, GET /api/notes/{id}/entities	Doctor / Admin
Medical Images	POST /api/images/analyze, GET /api/images/{id}	Doctor / Admin
Chatbot (NEW)	POST /api/chatbot/query, POST /api/chatbot/session, GET /api/chatbot/history	Doctor Only
Voice Dictation (NEW)	POST /api/voice/transcribe	Doctor Only
Drug Interactions (NEW)	POST /api/drugs/check-interactions	Doctor Only
Literature Search (NEW)	POST /api/literature/search	Doctor Only
Appointments	GET/POST/PUT/DELETE /api/appointments	Doctor / Patient / Admin
Prescriptions	GET/POST/PUT /api/prescriptions	Doctor / Admin
Chat	SignalR Hub /api/chat, GET /api/chat/history/{userId}	Doctor / Patient / Admin
Feedback	POST /api/feedback, GET /api/feedback	Patient / Admin
Audit Log	GET /api/audit, GET /api/audit/{userId}	Admin Only
Analytics	GET /api/analytics/dashboard, /predictions, /users	Admin Only
Docker Status (NEW)	GET /api/system/containers, /health	Admin Only

Table 11: MAPS API Endpoint Catalog (Including New Chatbot and Docker Endpoints)

13. Scope and Expected Outcomes

13.1 In Scope

- Three-role system: Admin, Doctor, Patient with hierarchical access control.
- Multi-disease prediction engine covering Diabetes, Heart Disease, Pneumonia, Brain Tumour, and Skin Cancer.
- Three prediction modalities: structured dataset input, medical image upload, and text prompt via Ollama LLM.
- AI Personalized Chatbot for doctors with four input modalities: text, image, voice, and document.
- RAG-enhanced chatbot context using pgvector for vector similarity search against patient health records.
- Voice-to-Text clinical dictation using Whisper.cpp with medical term correction.
- Drug Interaction AI Checker with severity classification and patient-specific risk assessment.

- Medical Literature Search using RAG architecture for clinical guidelines and drug monographs.
- Docker containerization with Docker Compose orchestration for one-command deployment.
- Hybrid deployment: WinForms desktop client and ASP.NET Core MVC web client sharing a single backend API.
- Real-time SignalR chat with file sharing supporting images, PDF, and DOCX attachments.
- Mandatory post-consultation feedback with Admin analytics dashboard and NLP sentiment analysis.
- First-run registration lock for Admin and Doctor roles with double-layer UI and API enforcement.
- JWT authentication and role-based API authorisation with short-lived access tokens and refresh token rotation.
- PDF report generation per patient and per prediction using QuestPDF (MIT-licensed, free).
- PostgreSQL database with Entity Framework Core ORM, pgvector extension, and full audit trails.
- NLP-driven clinical note summarization and medical entity extraction with F1 score above 0.80.
- ONNX-based medical image classification pipeline with CNN model inference in C# runtime.
- Continuous risk assessment with four-tier urgency classification and proactive alerting.
- All free and open-source technology stack with zero licensing costs.
- Grafana Loki + Serilog for structured logging, monitoring, and alerting.
- MinIO for self-hosted S3-compatible medical image and document storage.

13.2 Out of Scope (Initial Release)

- Native mobile application (iOS / Android).
- Integration with hospital PACS or HL7/FHIR electronic health record systems.
- Online payment or insurance billing modules.
- Training of deep learning models (models pre-trained in Python; ONNX inference only in C#).
- Kubernetes orchestration (Docker Compose is sufficient for the initial release scope).

13.3 Expected Outcomes

A fully functional hybrid C# medical AI prediction system deployable on a local server or clinic network via a single Docker Compose command is the primary expected outcome. This includes multi-disease prediction capability covering Diabetes, Heart Disease, Pneumonia, Brain Tumour, and Skin Cancer through three input modalities; a personalized AI chatbot for doctors supporting text, image, voice, and document inputs with RAG-enhanced clinical context; voice-to-text clinical dictation for hands-free note creation; intelligent drug interaction checking with patient-specific severity assessment; medical literature search grounded in clinical guidelines through RAG architecture; reduction in diagnostic workload for doctors through AI-assisted risk scoring with confidence percentages and clinical context; a secure real-time communication channel between doctors and patients with file sharing and permanently archived records; a compulsory feedback mechanism providing administrators with reliable structured doctor performance data; a secure onboarding process preventing unauthorized account creation; professional PDF report generation using QuestPDF; complete containerization with Docker Compose for consistent deployment; and a scalable maintainable codebase structured around the MVC pattern ready for future additions such as a public-facing patient portal or mobile client.

14. Conclusion

The MAPS platform represents a comprehensive approach to AI-augmented healthcare that prioritizes clinical safety, professional oversight, and patient-centered care. By implementing a strict role hierarchy where Administrators govern the platform, Doctors leverage advanced AI tools including a personalized multi-modal chatbot for clinical decision support, and Patients access health information through a mediated secure portal, MAPS ensures that AI serves as an assistive technology under human clinical control rather than an autonomous decision-maker.

The enhanced Doctor Module, with its sixteen features including the flagship AI Personalized Chatbot, Voice-to-Text Dictation, Drug Interaction AI Checker, and Medical Literature Search, transforms the platform from a simple prediction tool into a comprehensive AI-augmented clinical workspace. The chatbot's multi-modal input support (text, image, voice, document) combined with RAG-enhanced context retrieval provides doctors with an intelligent clinical assistant that is deeply integrated with patient health records and grounded in factual medical data, significantly reducing hallucinations and improving clinical relevance of AI suggestions.

The migration to an entirely free and open-source technology stack (PostgreSQL replacing SQL Server, QuestPDF replacing iTextSharp, MinIO replacing Azure Blob, Ollama replacing Groq API, Grafana Loki replacing Seq) demonstrates that enterprise-grade healthcare technology can be built without licensing costs or vendor dependency. Docker containerization with Docker Compose orchestration ensures consistent deployment, network isolation, and service manageability, reducing the operational complexity of running a multi-service platform to a single command.

The platform's modular architecture supports future expansion including wearable device integration for continuous vital sign monitoring, telemedicine video consultations, predictive population health analytics, Kubernetes orchestration for horizontal scaling, and integration with national electronic health record systems. MAPS demonstrates that AI in healthcare can be both powerful and responsible, combining cutting-edge machine learning with the clinical governance framework necessary to ensure patient safety and professional accountability.